



Diagrama de Classes UML

NOMES	RA
João Paulo Nunes Andrade	25002703
Caique Andraus	25006881
Gustavo Antoniazi	25007671
Pedro Carvalho	21005320

PROFESSOR	EMAIL
Leonardo Rezende Costa	leonardo.costa@puc-campinas.edu.br

1. INTRODUÇÃO

A Programação Orientada a Objetos (POO) é um paradigma de desenvolvimento de software fundado na abstração do mundo real por meio da criação de estruturas denominadas classes. A linguagem de modelagem unificada (*Unified Modeling Language* - UML) estabelece uma notação gráfica padronizada para documentar, projetar e especificar tais estruturas antes ou durante a codificação.

Este relatório tem como objetivo apresentar a documentação e o Diagrama de Classes UML correspondente à entidade **ContaBancaria**, desenvolvida na aula teórica anterior. A modelagem abrange a especificação exata de seus atributos de estado, métodos de comportamento e visibilidade de acesso (encapsulamento).

2. FUNDAMENTAÇÃO TEÓRICA

2.1. Abstração e Encapsulamento

A abstração permite selecionar os atributos e comportamentos indispensáveis de uma entidade do mundo real no contexto do sistema. Para assegurar a integridade dos dados financeiros, aplica-se o Encapsulamento, técnica que oculta a implementação interna do objeto e restringe o acesso direto aos seus dados.

- Visibilidade Privada (-): Atributos declarados como **private** só podem ser acessados ou alterados internamente pelos próprios métodos da classe.
- Visibilidade Pública (+): Métodos declarados como **public** compõem a interface da classe, permitindo a interação com o mundo externo de forma controlada.

2.2. Sintaxe Notacional do Diagrama de Classes UML

A representação de uma classe no diagrama UML é subdividida em três compartimentos horizontais:

1. Nome da Classe: Identificador centralizado na parte superior (**ContaBancaria**).
2. Atributos: Declaração no formato **visibilidade nomeAtributo : Tipo**.
3. Métodos: Declaração no formato **visibilidade nomeMetodo(parametro : Tipo) : TipoRetorno**.

3. DESCRIÇÃO DA CLASSE

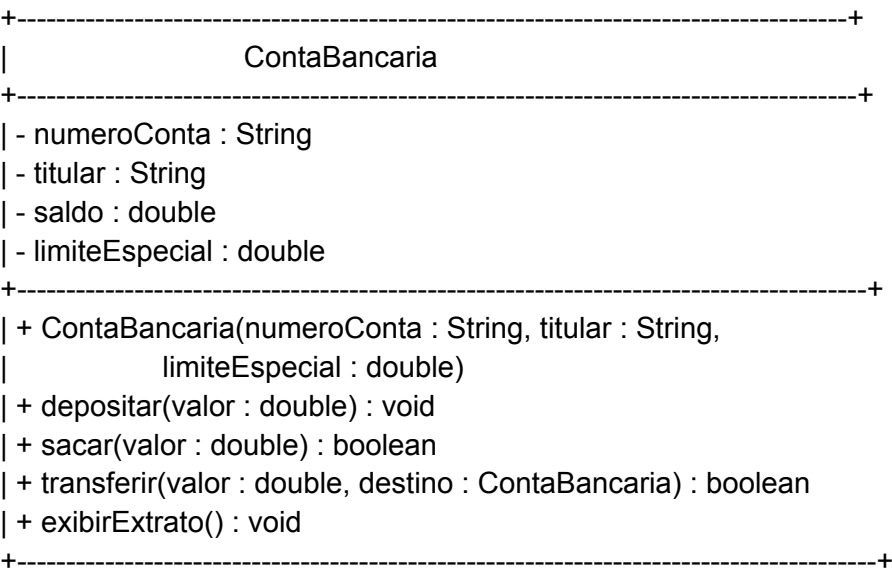
A classe **ContaBancaria** modela o comportamento de uma conta corrente com suporte a limite de crédito pré-aprovado.

3.1. Atributos

Visibilidade	Atributo	Tipo de Dado	Descrição / Função
private (-)	numeroConta	String	Identificador único da conta bancária.
private (-)	titular	String	Nome completo do proprietário da conta.
private (-)	saldo	double	Montante financeiro atualmente disponível.
private (-)	limiteEspecial	double	Crédito pré-aprovado para uso emergencial.

4. DIAGRAMA DE CLASSES UML

4.1. Versão Estrutural Padrão



4.2. Versão Completa

```
+-----+
|           ContaBancaria           |
+-----+
| - numeroConta : String             |
| - titular : String                 |
| - saldo : double                   |
| - limiteEspecial : double          |
+-----+
| + ContaBancaria(numeroConta : String, titular : String, |
|           limiteEspecial : double)                       |
| + depositar(valor : double) : void                       |
| + sacar(valor : double) : boolean                        |
| + transferir(valor : double, destino : ContaBancaria) : boolean |
| + exibirExtrato() : void                                  |
| + getNumeroConta() : String                               |
| + getTitular() : String                                   |
| + getSaldo() : double                                     |
| + getLimiteEspecial() : double                            |
| + setLimiteEspecial(limiteEspecial : double) : void      |
+-----+
```

5. REGRAS DE NEGÓCIO E ARQUITETURA DE SOFTWARE

1. Garantia de Consistência nos Débitos: O método **sacar** evita a ocorrência de saldos inconsistentes sem cobertura, pois valida a condição $valor \leq (saldo + limiteEspecial)$ antes de alterar o atributo **saldo**.
2. Reaproveitamento de Código (Modularidade): O método **transferir** não duplica a lógica de validação financeira; ele reutiliza as operações **sacar()** na própria instância e **depositar()** na instância **destino**.
3. Restrição de Acesso: O uso do modificador **private** impede que partes externas da aplicação realizem atribuições diretas ilegais (como modificar o valor do saldo arbitrariamente).

6. CONCLUSÃO

A representação da classe **ContaBancaria** no formato do Diagrama de Classes UML permitiu mapear claramente os conceitos teóricos de POO exercitados na prática. A formalização gráfica explicita como os modificadores de visibilidade protegem os dados e como as assinaturas dos métodos expõem apenas os comportamentos necessários e seguros para o funcionamento do sistema.